

Water Meter YOLO Model Training Guide

Author:

**Gabriel BAZIRAMWABO,
Rwanda Coding Academy | Benax Technologies, May 2026**

Part 3: Training, Retraining, and Prediction

Related Guides:

Part 1: Water Meter Dataset Collection Guide

Part 2: Water Meter Dataset Labeling Guide

Introduction

This guide explains the complete workflow for preparing, training, retraining, and testing a YOLO-based object detection model for an AI-based water meter reading system. This document is Part 3 of the Water Meter AI Project series:

- Part 1: Water Meter Dataset Collection Guide
- Part 2: Water Meter Dataset Labeling Guide
- **Part 3: YOLO Model Training, Retraining, and Prediction**

YOLO (You Only Look Once) is a real-time object detection model capable of detecting multiple objects in a single image quickly and efficiently. The model is trained to detect:

- meter: the full physical water meter
- window: the rectangular reading/display area
- digits 0–9: visible digits inside the reading window
- u: unclear or unreadable digits

Objective

Prepare, train, retrain, and test a YOLO object detection model for an AI-based water meter reading system.

1. Install Required Packages

```
pip install ultralytics  
pip install opencv-python
```

To verify type: `yolo`

2. Raw Dataset Structure

Make sure all labeled images and their labels are arranged as follows:

```
raw_dataset/  
├── images/  
└── labels/
```

Example:

```
raw_dataset/  
├── images/  
│   ├── wm_0001.jpg  
│   ├── wm_0002.jpg  
│   └── ...  
└── labels/  
    ├── wm_0001.txt  
    ├── wm_0002.txt  
    └── ...
```

Important:

- Every image must have a matching **.txt** label file.
- Image and label filenames must match.
- The **.txt** file must use YOLO format.
- Unsupported or junk files are ignored automatically.

3. Prepare YOLO Dataset

Run:

```
python3 scripts/02_prepare_dataset.py raw_dataset dataset
```

Or

```
python3 -m scripts.02_prepare_dataset raw_dataset dataset
```

The full source code of **02_prepare_dataset.py** is provided in the Appendix section of this guide.

This script automatically:

- collects labeled data
- ignores junk/system files
- validates YOLO labels
- removes invalid pairs
- renames files safely
- splits train/val/test
- creates **data.yaml**

Output:

```
dataset/
├── images/
│   ├── train/
│   ├── val/
│   └── test/
├── labels/
│   ├── train/
│   ├── val/
│   └── test/
├── reports/
└── data.yaml
```

4. Class Order

The dataset uses this class order:

```
names:
  0: meter
  1: window
  2: 0
  3: 1
  4: 2
  5: 3
  6: 4
  7: 5
  8: 6
  9: 7
  10: 8
  11: 9
  12: u
```

Important: The class IDs inside label files must match this exact order.

5. Check Dataset Split

Verify image counts:

```
find dataset/images/train -type f | wc -l
find dataset/images/val -type f | wc -l
find dataset/images/test -type f | wc -l
```

Verify label counts:

```
find dataset/labels/train -type f | wc -l
find dataset/labels/val -type f | wc -l
find dataset/labels/test -type f | wc -l
```

The number of images and labels must match.

6. Train YOLO

Run:

```
python3 scripts/03_train.py
Or
python3 -m scripts.03_train
```

The full source code of **03_train.py** is provided in the Appendix section of this guide.

The script automatically uses: **dataset/data.yaml**

Training outputs are saved in: **training_runs/**

Example: `training_runs/water_meter_yolov8n/`

Best model: `training_runs/.../weights/best.pt`

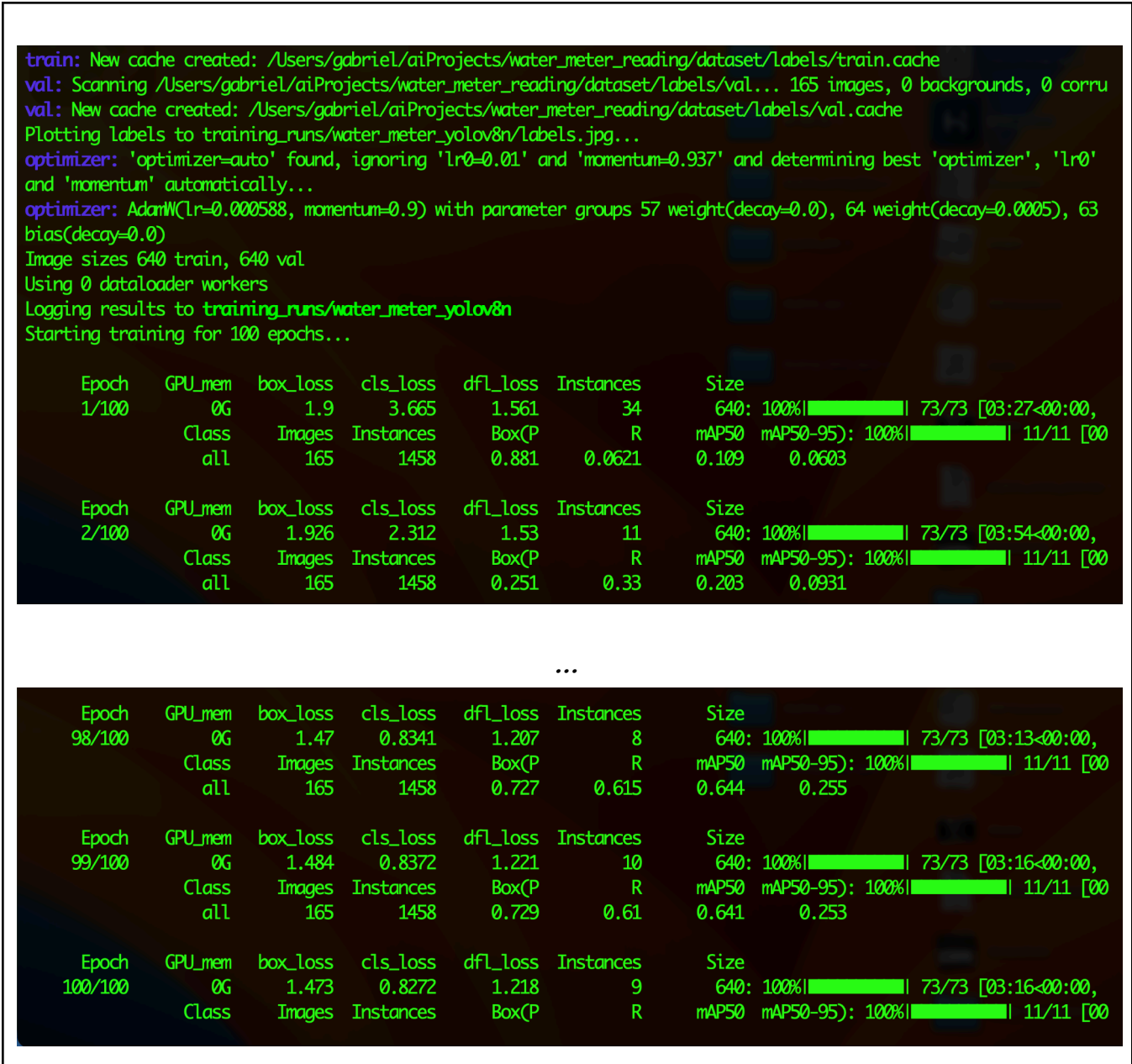


Image 1 & 2: Screenshots showing the beginning and completion of the YOLO model training process.

```

100 epochs completed in 8.431 hours.
Optimizer stripped from training_runs/water_meter_yolov8n/weights/last.pt, 6.2MB
Optimizer stripped from training_runs/water_meter_yolov8n/weights/best.pt, 6.2MB

Validating training_runs/water_meter_yolov8n/weights/best.pt...
Ultralytics 8.3.87 Python-3.10.20 torch-2.6.0 CPU (Apple M1)
Model summary (fused): 72 layers, 3,008,183 parameters, 0 gradients, 8.1 GFLOPs

```

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	11/11 [00
all	165	1458	0.723	0.639	0.671	0.274	
meter	164	164	0.942	0.951	0.944	0.687	
window	164	164	0.725	0.852	0.752	0.314	
0	151	331	0.822	0.755	0.767	0.292	
1	78	98	0.682	0.635	0.653	0.229	
2	60	80	0.74	0.65	0.675	0.225	
3	70	98	0.657	0.627	0.629	0.241	
4	74	83	0.795	0.653	0.724	0.275	
5	82	99	0.735	0.626	0.695	0.236	
6	65	70	0.796	0.457	0.588	0.226	
7	50	54	0.577	0.759	0.713	0.301	
8	57	66	0.811	0.621	0.719	0.254	
9	58	71	0.773	0.576	0.673	0.233	
u	56	80	0.349	0.15	0.19	0.0554	

```

Speed: 0.8ms preprocess, 222.9ms inference, 0.0ms loss, 0.8ms postprocess per image
Results saved to training_runs/water_meter_yolov8n

```

Image 3: Screenshot showing the YOLO training evaluation metrics including Precision, Recall, mAP50, and mAP50-95.

7. Retrain from Latest Best Model

Run:

```
python3 scripts/05_retrain.py
```

Or

```
python3 -m scripts.05_retrain
```

The full source code of **05_retrain.py** is provided in the Appendix section of this guide.

The script automatically:

- finds latest **best.pt**
- continues training from it
- creates a new training run

Example:

```
training_runs/water_meter_retrain_20260510_161200/
```

8. Predict on Test Images

Run:

```
python3 scripts/04_predict.py  
Or  
python3 -m scripts.04_predict
```

The full source code of **04_predict.py** is provided in the Appendix section of this guide.

The script automatically:

- finds latest **best.pt**
- predicts on **dataset/images/test**
- saves annotated images
- saves prediction **.txt** files
- saves confidence scores

Outputs: **prediction_outputs/test_predictions/**

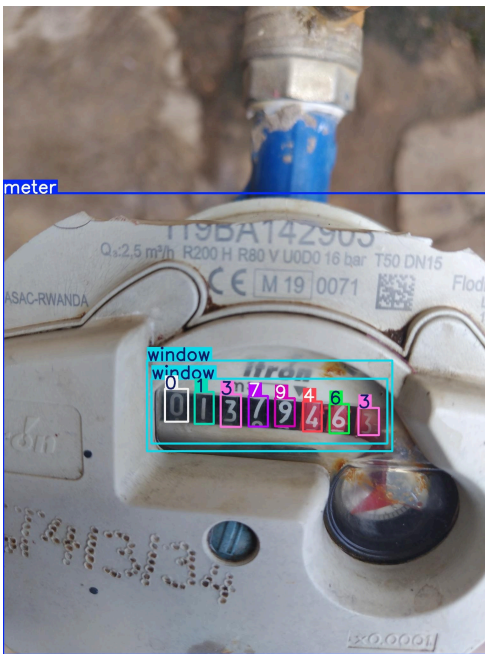


Image 5

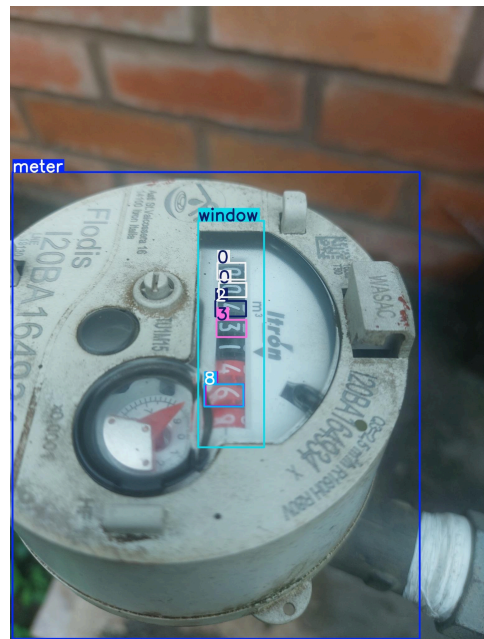


Image 6

Image 5: Screenshot showing successful detection of the water meter, reading window, and individual digits during YOLO prediction.

Image 6: Screenshot showing some prediction errors during YOLO inference, including digit confusion where a digit 6 was incorrectly detected as 8. Image 6 also shows that the model performs better when the water meter image is captured with minimal rotation. Strong rotation can make digit detection harder and may cause digit confusion, such as detecting 6 as 8.

Please note that for Image 5 and Image 6, digits detected outside the reading window are ignored according to the custom prediction logic implemented in the script.

9. Prediction Logic

The system follows this logic:

Image

↓

YOLO prediction

↓

Find reading window

↓

Ignore digits outside the reading window

↓

Keep digits inside the reading window only

↓

Sort digits left to right

↓

Create final meter reading

↓

If unknown digit exists: Ask user to retake image

10. Common Problems

#	Problem	Cause	Fix
01	data.yaml images not found	Wrong dataset path.	Use absolute dataset path inside <i>dataset/data.yaml</i>
02	OpenCV image too large	Extremely large or corrupted image.	Remove or resize the image.
03	Wrong class names during prediction	Mismatch between “label class IDs” and “data.yaml class order”	Keep class order consistent.

11. Important Notes

- Tight bounding boxes improve accuracy.
- Poor labels reduce model quality.
- Window labels must cover only the reading window.
- Digits outside the window should be ignored during reading.
- Use u only for unclear digits.
- More diverse images improve generalization.
- If a digit is too unclear or ambiguous, it is better to leave it unlabeled than to force an incorrect label that may confuse the model.
- If training results are poor, review and optimize the labeling quality before retraining the model.

12. Final Goal

The final system should:

- detect the water meter
- detect the reading window
- detect digits correctly
- ignore irrelevant digits
- order digits left to right
- generate meaningful readings
- detect unclear digits
- guide users to capture better images

13. Next Step

Optimize the dataset and labeling process before deployment.

Actions:

- Improve labeling quality.
- Retune '**`scripts/01_label_images.py`**' if needed. *The full source code of **`01_label_images.py`** is provided in the Appendix section of this guide.*
- Remove bad images from the dataset.
- Avoid heavily rotated images.
- Prefer images where the reading window is close to horizontal.
- Retrain the model until the metrics are acceptable.
- Start inference testing using a webcam on live water meters.
- After successful live testing, deploy the model to mobile devices.

Appendix: Project Scripts

This appendix contains the main scripts used in the Rwanda Water Meter Reading System project.

Project Repository: <https://github.com/benax-rw/rwanda-water-meter-reading-system>

The repository contains the complete workflow for:

- dataset collection
- image labeling
- dataset preparation
- YOLO training
- retraining
- prediction
- reading reconstruction

Appendix A: scripts/01_label_images.py

Purpose:

Prepare and label water meter images manually using a graphical labeling tool.

Main tasks:

- open a folder of water meter images
- draw bounding boxes around the meter
- draw bounding boxes around the reading window
- draw bounding boxes around digits
- label unclear digits as **u**
- rotate images when needed
- remove poor or unusable images
- save labels in YOLO format

Script:

```
#!/usr/bin/env python3
"""
01_label_images.py

Water Meter YOLO Labeler
"""

import os
import sys
import shutil
from dataclasses import dataclass
from typing import List, Optional, Tuple

import cv2
from PySide6.QtCore import QPointF, QRectF, Qt
from PySide6.QtGui import QAction, QColor, QImage, QKeySequence, QPainter, QPen, QPixmap, QBrush
from PySide6.QtWidgets import (
    QApplication,
    QFileDialog,
```

```

QLabel,
QMainWindow,
QPushButton,
QHBoxLayout,
QVBoxLayout,
QWidget,
QSizePolicy,
QInputDialog,
QDialog,
)

CLASS_NAMES = [
    "meter", "window",
    "0", "1", "2", "3", "4", "5", "6", "7", "8", "9",
    "unknown",
]

CLASS_COLORS = {
    0: QColor(0, 255, 0),
    1: QColor(255, 0, 0),
    2: QColor(255, 255, 0),
    3: QColor(255, 165, 0),
    4: QColor(0, 255, 255),
    5: QColor(255, 0, 255),
    6: QColor(128, 255, 0),
    7: QColor(0, 128, 255),
    8: QColor(255, 128, 128),
    9: QColor(128, 0, 255),
    10: QColor(200, 200, 255),
    11: QColor(255, 200, 200),
    12: QColor(180, 180, 180),
}

IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

HANDLE_SIZE = 10
MIN_BOX_SIZE = 5

def list_image_files(folder: str) -> List[str]:
    return sorted([
        f for f in os.listdir(folder)
        if os.path.isfile(os.path.join(folder, f))
        and os.path.splitext(f)[1].lower() in IMAGE_EXTENSIONS
    ])

def unique_destination_path(destination_path: str) -> str:
    if not os.path.exists(destination_path):
        return destination_path

    folder = os.path.dirname(destination_path)
    filename = os.path.basename(destination_path)
    stem, ext = os.path.splitext(filename)

    counter = 1

    while True:
        new_path = os.path.join(folder, f"{stem}_removed_{counter}{ext}")
        if not os.path.exists(new_path):
            return new_path
        counter += 1

def rotate_image_keep_size_crop_edges(image, angle_degrees: float):
    h, w = image.shape[:2]
    center = (w / 2.0, h / 2.0)

    matrix = cv2.getRotationMatrix2D(center, angle_degrees, 1.0)

    return cv2.warpAffine(
        image,
        matrix,
        (w, h),

```

```

        flags=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_CONSTANT,
        borderValue=(0, 0, 0),
    )

def crop_image(image, x1, y1, x2, y2):
    h, w = image.shape[:2]

    x_min = int(max(0, min(x1, x2)))
    y_min = int(max(0, min(y1, y2)))
    x_max = int(min(w, max(x1, x2)))
    y_max = int(min(h, max(y1, y2)))

    if x_max <= x_min or y_max <= y_min:
        return None

    return image[y_min:y_max, x_min:x_max]

def safe_message(parent, title: str, message: str, button_text: str = "OK") -> None:
    dialog = QDialog(parent)
    dialog.setWindowTitle(title)
    dialog.setModal(True)
    dialog.resize(520, 170)

    label = QLabel(message)
    label.setWordWrap(True)

    ok_btn = QPushButton(button_text)
    ok_btn.clicked.connect(dialog.accept)

    buttons = QHBoxLayout()
    buttons.addStretch()
    buttons.addWidget(ok_btn)

    layout = QVBoxLayout()
    layout.addWidget(label)
    layout.addStretch()
    layout.addLayout(buttons)

    dialog.setLayout(layout)
    dialog.exec()

def safe_confirm(parent, title: str, message: str, yes_text: str = "Yes", no_text: str = "Cancel") ->
bool:
    dialog = QDialog(parent)
    dialog.setWindowTitle(title)
    dialog.setModal(True)
    dialog.resize(560, 200)

    label = QLabel(message)
    label.setWordWrap(True)

    yes_btn = QPushButton(yes_text)
    no_btn = QPushButton(no_text)

    yes_btn.clicked.connect(dialog.accept)
    no_btn.clicked.connect(dialog.reject)

    buttons = QHBoxLayout()
    buttons.addStretch()
    buttons.addWidget(yes_btn)
    buttons.addWidget(no_btn)

    layout = QVBoxLayout()
    layout.addWidget(label)
    layout.addStretch()
    layout.addLayout(buttons)

    dialog.setLayout(layout)

    return dialog.exec() == QDialog.Accepted

```

```

@dataclass
class Box:
    class_id: int
    x1: float
    y1: float
    x2: float
    y2: float

    def copy(self) -> "Box":
        return Box(self.class_id, self.x1, self.y1, self.x2, self.y2)

    def normalized(self, img_w: int, img_h: int) -> Tuple[float, float, float, float]:
        x_min = min(self.x1, self.x2)
        y_min = min(self.y1, self.y2)
        x_max = max(self.x1, self.x2)
        y_max = max(self.y1, self.y2)

        x_center = ((x_min + x_max) / 2.0) / img_w
        y_center = ((y_min + y_max) / 2.0) / img_h
        width = (x_max - x_min) / img_w
        height = (y_max - y_min) / img_h

        return x_center, y_center, width, height

    @staticmethod
    def from_normalized(class_id, x_center, y_center, width, height, img_w, img_h) -> "Box":
        bw = width * img_w
        bh = height * img_h
        cx = x_center * img_w
        cy = y_center * img_h

        return Box(
            class_id,
            cx - bw / 2.0,
            cy - bh / 2.0,
            cx + bw / 2.0,
            cy + bh / 2.0,
        )

    def rect(self):
        return (
            min(self.x1, self.x2),
            min(self.y1, self.y2),
            max(self.x1, self.x2),
            max(self.y1, self.y2),
        )

    def contains(self, x, y) -> bool:
        x_min, y_min, x_max, y_max = self.rect()
        return x_min <= x <= x_max and y_min <= y <= y_max

    def width(self):
        return abs(self.x2 - self.x1)

    def height(self):
        return abs(self.y2 - self.y1)

    def is_too_small(self, min_size: int = MIN_BOX_SIZE) -> bool:
        return self.width() < min_size or self.height() < min_size

    def move(self, dx, dy, img_w, img_h) -> None:
        x_min, y_min, x_max, y_max = self.rect()
        box_w = x_max - x_min
        box_h = y_max - y_min

        new_x_min = max(0, min(img_w - box_w, x_min + dx))
        new_y_min = max(0, min(img_h - box_h, y_min + dy))

        self.x1 = new_x_min
        self.y1 = new_y_min
        self.x2 = new_x_min + box_w
        self.y2 = new_y_min + box_h

    def clamp(self, img_w, img_h) -> None:

```

```

self.x1 = max(0, min(img_w - 1, self.x1))
self.y1 = max(0, min(img_h - 1, self.y1))
self.x2 = max(0, min(img_w - 1, self.x2))
self.y2 = max(0, min(img_h - 1, self.y2))

class ImageCanvas(QLabel):
    def __init__(self, parent=None):
        super().__init__(parent)

        self.setMouseTracking(True)
        self.setAlignment(Qt.AlignCenter)
        self.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)

        self.main_window = None

        self.base_pixmap = None
        self.display_pixmap = None

        self.scale = 1.0
        self.offset_x = 0
        self.offset_y = 0

        self.mode = "idle"
        self.crop_mode_enabled = False
        self.resize_handle = None

        self.start_widget_point = None
        self.current_widget_point = None
        self.last_img_point = None

        self.original_box_before_edit = None

    def set_main_window(self, window):
        self.main_window = window

    def set_image(self, qpixmap: QPixmap):
        self.base_pixmap = qpixmap
        self.update_scaled_pixmap()

    def clear_image(self):
        self.base_pixmap = None
        self.display_pixmap = None
        self.clear()

    def resizeEvent(self, event):
        super().resizeEvent(event)
        self.update_scaled_pixmap()

    def update_scaled_pixmap(self):
        if self.base_pixmap is None:
            self.clear()
            return

        label_w = max(1, self.width())
        label_h = max(1, self.height())

        self.display_pixmap = self.base_pixmap.scaled(
            label_w,
            label_h,
            Qt.KeepAspectRatio,
            Qt.SmoothTransformation,
        )

        self.offset_x = (label_w - self.display_pixmap.width()) // 2
        self.offset_y = (label_h - self.display_pixmap.height()) // 2
        self.scale = self.display_pixmap.width() / self.base_pixmap.width()

        canvas = QPixmap(label_w, label_h)
        canvas.fill(Qt.black)

        painter = QPainter(canvas)
        painter.drawPixmap(self.offset_x, self.offset_y, self.display_pixmap)

        if self.main_window is not None:

```

```

        self.main_window.draw_boxes(painter, self.scale, self.offset_x, self.offset_y)

    if self.mode in ("drawing", "crop") and self.start_widget_point and self.current_widget_point:
        if self.mode == "crop":
            draft_color = QColor(0, 255, 255)
        else:
            draft_color = CLASS_COLORS.get(
                self.main_window.current_class_id,
                QColor(255, 255, 255),
            )

        painter.setPen(QPen(draft_color, 2, Qt.DashLine))
        painter.setBrush(Qt.NoBrush)

        rect = QRectF(self.start_widget_point, self.current_widget_point).normalized()
        painter.drawRect(rect)

    painter.end()
    self.setPixmap(canvas)

def widget_to_image(self, point: QPointF):
    if self.base_pixmap is None or self.display_pixmap is None:
        return None

    x = point.x() - self.offset_x
    y = point.y() - self.offset_y

    if x < 0 or y < 0 or x > self.display_pixmap.width() or y > self.display_pixmap.height():
        return None

    return x / self.scale, y / self.scale

def mousePressEvent(self, event):
    if self.base_pixmap is None or self.main_window is None:
        return

    if event.button() != Qt.LeftButton:
        return

    widget_pos = QPointF(event.position())
    img_pos = self.widget_to_image(widget_pos)

    if img_pos is None:
        return

    self.start_widget_point = widget_pos
    self.current_widget_point = widget_pos
    self.last_img_point = img_pos

    if self.crop_mode_enabled:
        self.mode = "crop"
        self.update_scaled_pixmap()
        return

    handle = self.main_window.get_handle_at(*img_pos)

    if handle is not None:
        self.main_window.push_undo_state()
        self.mode = "resizing"
        self.resize_handle = handle[1]
        self.main_window.selected_box_index = handle[0]
        self.original_box_before_edit = self.main_window.bboxes[handle[0]].copy()
        self.update_scaled_pixmap()
        return

    border_box_index = self.main_window.get_border_at(*img_pos)

    if border_box_index is not None:
        self.main_window.push_undo_state()
        self.mode = "moving"
        self.main_window.selected_box_index = border_box_index
        self.original_box_before_edit = self.main_window.bboxes[border_box_index].copy()
        self.update_scaled_pixmap()
        return

```

```

        clicked_box_index = self.main_window.find_box_at(*img_pos)
        self.main_window.selected_box_index = clicked_box_index

        self.mode = "drawing"
        self.resize_handle = None
        self.original_box_before_edit = None

        self.update_scaled_pixmap()

def mouseMoveEvent(self, event):
    if self.main_window is None or self.base_pixmap is None:
        return

    widget_pos = QPointF(event.position())
    img_pos = self.widget_to_image(widget_pos)

    if self.mode in ("drawing", "crop"):
        self.current_widget_point = widget_pos
        self.update_scaled_pixmap()
        return

    if img_pos is None:
        return

    if self.mode == "moving":
        if self.main_window.selected_box_index is None or self.last_img_point is None:
            return

        dx = img_pos[0] - self.last_img_point[0]
        dy = img_pos[1] - self.last_img_point[1]

        box = self.main_window.bboxes[self.main_window.selected_box_index]
        box.move(dx, dy, self.main_window.current_image_w, self.main_window.current_image_h)

        self.last_img_point = img_pos
        self.update_scaled_pixmap()
        self.main_window.update_status(extra="Moving box...")
        return

    if self.mode == "resizing":
        if self.main_window.selected_box_index is None:
            return

        box = self.main_window.bboxes[self.main_window.selected_box_index]
        x, y = img_pos

        if self.resize_handle == "tl":
            box.x1, box.y1 = x, y
        elif self.resize_handle == "tr":
            box.x2, box.y1 = x, y
        elif self.resize_handle == "bl":
            box.x1, box.y2 = x, y
        elif self.resize_handle == "br":
            box.x2, box.y2 = x, y

        box.clamp(self.main_window.current_image_w, self.main_window.current_image_h)

        self.update_scaled_pixmap()
        self.main_window.update_status(extra="Resizing box...")
        return

def mouseReleaseEvent(self, event):
    if self.base_pixmap is None or self.main_window is None:
        return

    if event.button() != Qt.LeftButton:
        return

    if self.mode == "crop":
        end_point = QPointF(event.position())

        start_img = self.widget_to_image(self.start_widget_point) if self.start_widget_point else None
        end_img = self.widget_to_image(end_point)

        self.mode = "idle"

```



```

        self.crop_mode_enabled = False

        if start_img is not None and end_img is not None:
            self.main_window.crop_current_image(start_img, end_img)
        else:
            self.main_window.update_status(extra="Crop cancelled.")

        self.start_widget_point = None
        self.current_widget_point = None
        self.last_img_point = None
        self.update_scaled_pixmap()
        return

    if self.mode == "drawing":
        end_point = QPointF(event.position())

        start_img = self.widget_to_image(self.start_widget_point) if self.start_widget_point else None
        end_img = self.widget_to_image(end_point)

        if start_img is not None and end_img is not None:
            x1, y1 = start_img
            x2, y2 = end_img

            new_box = Box(self.main_window.current_class_id, x1, y1, x2, y2)

            if not new_box.is_too_small():
                self.main_window.push_undo_state()
                self.main_window.bboxes.append(new_box)
                self.main_window.selected_box_index = len(self.main_window.bboxes) - 1
                self.main_window.update_status(extra="Box added.")
            else:
                self.main_window.update_status(extra="Ignored tiny box.")

        elif self.mode in ("moving", "resizing"):
            if self.main_window.selected_box_index is not None:
                box = self.main_window.bboxes[self.main_window.selected_box_index]
                box.clamp(self.main_window.current_image_w, self.main_window.current_image_h)

                if box.is_too_small():
                    if self.original_box_before_edit is not None:
                        self.main_window.bboxes[self.main_window.selected_box_index] =
self.original_box_before_edit
                    self.main_window.update_status(extra="Edit cancelled: box too small.")
                else:
                    self.main_window.update_status(extra="Box updated.")

        self.mode = "idle"
        self.resize_handle = None
        self.start_widget_point = None
        self.current_widget_point = None
        self.last_img_point = None
        self.original_box_before_edit = None

        self.update_scaled_pixmap()

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setWindowTitle("Water Meter YOLO Labeler")
        self.resize(1280, 850)

        self.image_dir = None
        self.label_dir = None
        self.image_files = []
        self.current_index = 0

        self.current_image = None
        self.current_image_w = 0
        self.current_image_h = 0

        self.bboxes = []
        self.selected_box_index = None
        self.current_class_id = 0

```

```

self.undo_stack = []

self.canvas = ImageCanvas()
self.canvas.set_main_window(self)

self.info_label = QLabel("Open an image folder to begin.")
self.info_label.setWordWrap(True)

open_btn = QPushButton("Open Folder")
open_btn.clicked.connect(self.open_folder)

prev_btn = QPushButton("Previous (P)")
prev_btn.clicked.connect(self.prev_image)

next_btn = QPushButton("Next (N)")
next_btn.clicked.connect(self.next_image)

save_btn = QPushButton("Save")
save_btn.clicked.connect(self.save_labels)

remove_btn = QPushButton("Remove Image")
remove_btn.clicked.connect(self.remove_current_image)

undo_btn = QPushButton("Undo")
undo_btn.clicked.connect(self.undo)

crop_btn = QPushButton("Crop Image")
crop_btn.clicked.connect(self.enable_crop_mode)

rot_left_btn = QPushButton("Rotate -15°")
rot_left_btn.clicked.connect(lambda: self.rotate_current_image(-15))

rot_right_btn = QPushButton("Rotate +15°")
rot_right_btn.clicked.connect(lambda: self.rotate_current_image(15))

rot_90_btn = QPushButton("Rotate 90°")
rot_90_btn.clicked.connect(lambda: self.rotate_current_image(90))

rot_custom_btn = QPushButton("Rotate Custom")
rot_custom_btn.clicked.connect(self.rotate_custom_angle)

top_bar = QHBoxLayout()
top_bar.addWidget(open_btn)
top_bar.addWidget(prev_btn)
top_bar.addWidget(next_btn)
top_bar.addWidget(save_btn)
top_bar.addWidget(remove_btn)
top_bar.addWidget(undo_btn)
top_bar.addWidget(crop_btn)
top_bar.addWidget(rot_left_btn)
top_bar.addWidget(rot_right_btn)
top_bar.addWidget(rot_90_btn)
top_bar.addWidget(rot_custom_btn)
top_bar.addStretch()

main_layout = QVBoxLayout()
main_layout.addLayout(top_bar)
main_layout.addWidget(self.canvas, stretch=1)
main_layout.addWidget(self.info_label)

central = QWidget()
central.setLayout(main_layout)
self.setCentralWidget(central)

self.create_shortcuts()
self.create_menu()
self.update_status()

def create_menu(self):
    open_action = QAction("Open Folder", self)
    open_action.setShortcut(QKeySequence.Open)
    open_action.triggered.connect(self.open_folder)

    save_action = QAction("Save", self)
    save_action.setShortcut(QKeySequence.Save)

```

```

save_action.triggered.connect(self.save_labels)

remove_action = QAction("Remove Image", self)
remove_action.setShortcut(QKeySequence("Ctrl+D"))
remove_action.triggered.connect(self.remove_current_image)

undo_action = QAction("Undo", self)
undo_action.setShortcut(QKeySequence.Undo)
undo_action.triggered.connect(self.undo)

crop_action = QAction("Crop Image", self)
crop_action.setShortcut(QKeySequence("C"))
crop_action.triggered.connect(self.enable_crop_mode)

file_menu = self.menuBar().addMenu("File")
file_menu.addAction(open_action)
file_menu.addAction(save_action)
file_menu.addAction(remove_action)

edit_menu = self.menuBar().addMenu("Edit")
edit_menu.addAction(undo_action)
edit_menu.addAction(crop_action)

def create_shortcuts(self):
    shortcuts = [
        ("OpenFolder", "Ctrl+O", self.open_folder),
        ("Save", "Ctrl+S", self.save_labels),
        ("Undo", "Ctrl+Z", self.undo),
        ("Next", "N", self.next_image),
        ("Prev", "P", self.prev_image),
        ("RemoveImage", "Ctrl+D", self.remove_current_image),
        ("CropImage", "C", self.enable_crop_mode),
        ("RotateMinus15", "Ctrl+Left", lambda: self.rotate_current_image(-15)),
        ("RotatePlus15", "Ctrl+Right", lambda: self.rotate_current_image(15)),
        ("Rotate90", "R", lambda: self.rotate_current_image(90)),
        ("RotateCustom", "Ctrl+R", self.rotate_custom_angle),
        ("Meter", "M", lambda: self.set_class(0)),
        ("Window", "W", lambda: self.set_class(1)),
        ("ZeroDigit", "0", lambda: self.set_class(2)),
        ("OneDigit", "1", lambda: self.set_class(3)),
        ("TwoDigit", "2", lambda: self.set_class(4)),
        ("ThreeDigit", "3", lambda: self.set_class(5)),
        ("FourDigit", "4", lambda: self.set_class(6)),
        ("FiveDigit", "5", lambda: self.set_class(7)),
        ("SixDigit", "6", lambda: self.set_class(8)),
        ("SevenDigit", "7", lambda: self.set_class(9)),
        ("EightDigit", "8", lambda: self.set_class(10)),
        ("NineDigit", "9", lambda: self.set_class(11)),
        ("UnknownDigit", "U", lambda: self.set_class(12)),
        ("Delete", "Delete", self.delete_selected_box),
        ("DeleteBackspace", "Backspace", self.delete_selected_box),
        ("ClearSelection", "Escape", self.clear_selection),
    ]

    for name, shortcut, slot in shortcuts:
        action = QAction(name, self)
        action.setShortcut(QKeySequence(shortcut))
        action.triggered.connect(slot)
        self.addAction(action)

def enable_crop_mode(self):
    if self.current_image is None:
        self.update_status(extra="Open an image first.")
        return

    self.canvas.crop_mode_enabled = True
    self.canvas.mode = "idle"
    self.selected_box_index = None
    self.canvas.update_scaled_pixmap()
    self.update_status(extra="Crop mode enabled. Draw a rectangle around the useful meter area.")

def open_folder(self):
    start_folder = self.image_dir if self.image_dir else os.getcwd()

    folder = QFileDialog.getExistingDirectory(

```

```

        self,
        "Select Image Folder",
        start_folder,
        QFileDialog.ShowDirsOnly | QFileDialog.DontUseNativeDialog,
    )

    if not folder:
        return

    image_files = list_image_files(folder)

    if not image_files:
        safe_message(
            self,
            "No Images",
            "This folder has no supported images.\n\nPlease open the folder that directly contains the\n\nimages.",
        )
        return

    self.image_dir = folder

    parent_dir = os.path.dirname(folder)
    self.label_dir = os.path.join(parent_dir, "labels")
    os.makedirs(self.label_dir, exist_ok=True)

    self.image_files = image_files
    self.current_index = 0

    self.load_current_image()
    self.update_status(extra=f"Images: {self.image_dir} | Labels: {self.label_dir}")

def current_label_path(self):
    if not self.label_dir or not self.image_files:
        return None

    stem = os.path.splitext(self.image_files[self.current_index])[0]
    return os.path.join(self.label_dir, f"{stem}.txt")

def push_undo_state(self):
    self.undo_stack.append([box.copy() for box in self.bboxes])
    if len(self.undo_stack) > 200:
        self.undo_stack.pop(0)

def undo(self):
    if not self.undo_stack:
        self.update_status(extra="Nothing to undo.")
        return

    self.bboxes = self.undo_stack.pop()
    self.selected_box_index = None
    self.canvas.update_scaled_pixmap()
    self.update_status(extra="Undid last action.")

def clear_selection(self):
    self.selected_box_index = None
    self.canvas.crop_mode_enabled = False
    self.canvas.update_scaled_pixmap()
    self.update_status(extra="Selection cleared.")

def load_current_image(self):
    if not self.image_dir or not self.image_files:
        return

    image_path = os.path.join(self.image_dir, self.image_files[self.current_index])
    image = cv2.imread(image_path)

    if image is None:
        self.canvas.clear_image()
        self.current_image = None
        self.current_image_w = 0
        self.current_image_h = 0
        self.bboxes = []
        self.selected_box_index = None
        self.update_status(extra=f"Failed to load image: {self.image_files[self.current_index]}")

```

```

        return

    self.current_image = image
    self.current_image_h, self.current_image_w = image.shape[:2]

    rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    qimage = QImage(
        rgb.data,
        self.current_image_w,
        self.current_image_h,
        rgb.strides[0],
        QImage.Format_RGB888,
    )

    pixmap = QPixmap.fromImage(qimage.copy())

    self.undo_stack = []
    self.bboxes = []
    self.selected_box_index = None
    self.canvas.crop_mode_enabled = False

    self.canvas.set_image(pixmap)
    self.load_labels()
    self.update_status()

def load_labels(self):
    self.bboxes = []
    self.selected_box_index = None

    label_path = self.current_label_path()

    if not label_path or not os.path.exists(label_path):
        self.canvas.update_scaled_pixmap()
        return

    try:
        with open(label_path, "r", encoding="utf-8") as f:
            for line in f:
                parts = line.strip().split()

                if len(parts) != 5:
                    continue

                class_id = int(parts[0])
                x_center, y_center, width, height = map(float, parts[1:])

                if class_id < 0 or class_id >= len(CLASS_NAMES):
                    continue

                box = Box.from_normalized(
                    class_id,
                    x_center,
                    y_center,
                    width,
                    height,
                    self.current_image_w,
                    self.current_image_h,
                )

                self.bboxes.append(box)

    except Exception as e:
        self.update_status(extra=f"Label load error: {e}")

    self.canvas.update_scaled_pixmap()

def save_labels(self):
    label_path = self.current_label_path()

    if not label_path:
        return

    try:
        with open(label_path, "w", encoding="utf-8") as f:

```

```

        for box in self.bboxes:
            x_center, y_center, width, height = box.normalized(
                self.current_image_w,
                self.current_image_h,
            )

            f.write(
                f"{box.class_id} "
                f"{x_center:.6f} "
                f"{y_center:.6f} "
                f"{width:.6f} "
                f"{height:.6f}\n"
            )

        self.update_status(extra="Saved.")

    except Exception as e:
        self.update_status(extra=f"Save error: {e}")

def clear_label_file(self):
    label_path = self.current_label_path()

    if label_path and os.path.exists(label_path):
        with open(label_path, "w", encoding="utf-8") as f:
            f.write("")

def crop_current_image(self, start_img, end_img):
    if not self.image_dir or not self.image_files:
        return

    image_path = os.path.join(self.image_dir, self.image_files[self.current_index])

    if self.bboxes:
        confirmed = safe_confirm(
            self,
            "Crop Image",
            (
                "This image already has labels.\n\n"
                "Cropping it will clear the current boxes because their positions will no longer"
                match.\n\n"
                "Continue?"
            ),
            yes_text="Yes, Crop",
            no_text="Cancel",
        )

        if not confirmed:
            self.update_status(extra="Cropping cancelled.")
            return

    image = cv2.imread(image_path)

    if image is None:
        self.update_status(extra=f"Could not read image: {image_path}")
        return

    x1, y1 = start_img
    x2, y2 = end_img

    cropped = crop_image(image, x1, y1, x2, y2)

    if cropped is None:
        self.update_status(extra="Invalid crop area.")
        return

    ok = cv2.imwrite(image_path, cropped)

    if not ok:
        self.update_status(extra=f"Could not save cropped image: {image_path}")
        return

    self.clear_label_file()
    self.load_current_image()
    self.update_status(extra="Image cropped and saved. Labels cleared.")

```

```

def remove_current_image(self):
    if not self.image_dir or not self.image_files:
        return

    image_name = self.image_files[self.current_index]
    image_path = os.path.join(self.image_dir, image_name)
    label_path = self.current_label_path()

    confirmed = safe_confirm(
        self,
        "Remove Image",
        (
            "Remove this image from the dataset?\n\n"
            f"{image_name}\n\n"
            "The image and its matching label file will be moved safely to:\n"
            "removed_images/ and removed_labels/"
        ),
        yes_text="Yes, Remove",
        no_text="Cancel",
    )

    if not confirmed:
        self.update_status(extra="Image removal cancelled.")
        return

    dataset_root = os.path.dirname(self.image_dir)

    removed_images_dir = os.path.join(dataset_root, "removed_images")
    removed_labels_dir = os.path.join(dataset_root, "removed_labels")

    os.makedirs(removed_images_dir, exist_ok=True)
    os.makedirs(removed_labels_dir, exist_ok=True)

    try:
        self.canvas.clear_image()
        self.current_image = None
        self.current_image_w = 0
        self.current_image_h = 0
        self.bboxes = []
        self.selected_box_index = None
        self.undo_stack = []

        if os.path.exists(image_path):
            image_destination = unique_destination_path(os.path.join(removed_images_dir, image_name))
            shutil.move(image_path, image_destination)

        if label_path and os.path.exists(label_path):
            label_destination = unique_destination_path(os.path.join(removed_labels_dir,
os.path.basename(label_path)))
            shutil.move(label_path, label_destination)

        del self.image_files[self.current_index]

        if not self.image_files:
            self.current_index = 0
            self.update_status(extra="Removed image. No images left.")
            return

        if self.current_index >= len(self.image_files):
            self.current_index = len(self.image_files) - 1

        self.load_current_image()
        self.update_status(extra=f"Removed image: {image_name}")

    except Exception as e:
        self.update_status(extra=f"Remove error: {e}")

def rotate_current_image(self, angle_degrees: float):
    if not self.image_dir or not self.image_files:
        return

    image_path = os.path.join(self.image_dir, self.image_files[self.current_index])

    if self.bboxes:
        confirmed = safe_confirm(

```

```

        self,
        "Rotate Image",
        (
            "This image already has labels.\n\n"
            "Rotating it will clear the current boxes because their positions will no longer
match.\n\n"
            "Continue?"
        ),
        yes_text="Yes, Rotate",
        no_text="Cancel",
    )

    if not confirmed:
        self.update_status(extra="Rotation cancelled.")
        return

    image = cv2.imread(image_path)

    if image is None:
        self.update_status(extra=f"Could not read image: {image_path}")
        return

    rotated = rotate_image_keep_size_crop_edges(image, angle_degrees)
    ok = cv2.imwrite(image_path, rotated)

    if not ok:
        self.update_status(extra=f"Could not save rotated image: {image_path}")
        return

    self.clear_label_file()
    self.load_current_image()
    self.update_status(extra=f"Image rotated by {angle_degrees}° and saved. Labels cleared.")

def rotate_custom_angle(self):
    angle, ok = QDialog.getDouble(
        self,
        "Rotate Image",
        "Enter rotation angle in degrees:",
        0.0,
        -360.0,
        360.0,
        2,
    )

    if ok:
        self.rotate_current_image(angle)

def next_image(self):
    if not self.image_files:
        return

    self.save_labels()

    if self.current_index < len(self.image_files) - 1:
        self.current_index += 1
        self.load_current_image()

def prev_image(self):
    if not self.image_files:
        return

    self.save_labels()

    if self.current_index > 0:
        self.current_index -= 1
        self.load_current_image()

def set_class(self, class_id: int):
    self.current_class_id = class_id
    self.canvas.crop_mode_enabled = False

    if self.selected_box_index is not None and 0 <= self.selected_box_index < len(self.bboxes):
        if self.bboxes[self.selected_box_index].class_id != class_id:
            self.push_undo_state()
            self.bboxes[self.selected_box_index].class_id = class_id

```



```

        self.canvas.update_scaled_pixmap()
        self.update_status(extra=f"Selected box changed to {CLASS_NAMES[class_id]}.")
        return

    self.update_status(extra=f"Current class: {CLASS_NAMES[class_id]}")

def delete_selected_box(self):
    if self.selected_box_index is None:
        self.update_status(extra="No box selected.")
        return

    if 0 <= self.selected_box_index < len(self.bboxes):
        self.push_undo_state()
        del self.bboxes[self.selected_box_index]
        self.selected_box_index = None
        self.canvas.update_scaled_pixmap()
        self.update_status(extra="Box deleted.")

def find_box_at(self, x, y):
    for i in range(len(self.bboxes) - 1, -1, -1):
        if self.bboxes[i].contains(x, y):
            return i
    return None

def get_handle_at(self, x, y):
    tolerance = max(6, HANDLE_SIZE / max(self.canvas.scale, 1e-6))

    for i in range(len(self.bboxes) - 1, -1, -1):
        box = self.bboxes[i]
        x_min, y_min, x_max, y_max = box.rect()

        handles = {
            "tl": (x_min, y_min),
            "tr": (x_max, y_min),
            "bl": (x_min, y_max),
            "br": (x_max, y_max),
        }

        for name, (hx, hy) in handles.items():
            if abs(x - hx) <= tolerance and abs(y - hy) <= tolerance:
                return i, name

    return None

def get_border_at(self, x, y):
    tolerance = max(6, HANDLE_SIZE / max(self.canvas.scale, 1e-6))

    for i in range(len(self.bboxes) - 1, -1, -1):
        box = self.bboxes[i]
        x_min, y_min, x_max, y_max = box.rect()

        near_left = abs(x - x_min) <= tolerance and y_min <= y <= y_max
        near_right = abs(x - x_max) <= tolerance and y_min <= y <= y_max
        near_top = abs(y - y_min) <= tolerance and x_min <= x <= x_max
        near_bottom = abs(y - y_max) <= tolerance and x_min <= x <= x_max

        if near_left or near_right or near_top or near_bottom:
            return i

    return None

def draw_boxes(self, painter: QPainter, scale: float, offset_x: int, offset_y: int):
    for i, box in enumerate(self.bboxes):
        x_min, y_min, x_max, y_max = box.rect()

        x1 = x_min * scale + offset_x
        y1 = y_min * scale + offset_y
        x2 = x_max * scale + offset_x
        y2 = y_max * scale + offset_y

        color = CLASS_COLORS.get(box.class_id, QColor(255, 255, 255))
        pen_width = 3 if i == self.selected_box_index else 2

        painter.setPen(QPen(color, pen_width))
        painter.setBrush(Qt.NoBrush)

```

```

painter.drawRect(QRectF(QPointF(x1, y1), QPointF(x2, y2)))

label_text = CLASS_NAMES[box.class_id]
text_width = 22 if label_text in list("0123456789") else len(label_text) * 8 + 12
text_height = 20
label_top = max(0, int(y1) - text_height)

painter.fillRect(int(x1), label_top, text_width, text_height, color)
painter.setPen(Qt.black)
painter.drawText(int(x1) + 5, label_top + 15, label_text)

if i == self.selected_box_index:
    painter.setBrush(QBrush(QColor(255, 255, 255)))
    painter.setPen(QPen(Qt.black, 1))

    handles = [
        QRectF(x1 - HANDLE_SIZE / 2, y1 - HANDLE_SIZE / 2, HANDLE_SIZE, HANDLE_SIZE),
        QRectF(x2 - HANDLE_SIZE / 2, y1 - HANDLE_SIZE / 2, HANDLE_SIZE, HANDLE_SIZE),
        QRectF(x1 - HANDLE_SIZE / 2, y2 - HANDLE_SIZE / 2, HANDLE_SIZE, HANDLE_SIZE),
        QRectF(x2 - HANDLE_SIZE / 2, y2 - HANDLE_SIZE / 2, HANDLE_SIZE, HANDLE_SIZE),
    ]

    for handle_rect in handles:
        painter.drawRect(handle_rect)

def update_status(self, extra: str = ""):
    shortcuts_text = (
        "M=meter | W=window | 0..9=digit classes | U=unknown | "
        "C=crop image | N=next | P=previous | R=rotate 90° | "
        "Ctrl+Left=-15° | Ctrl+Right+=15° | Ctrl+R=custom rotate | "
        "Border drag=move box | Corner drag=resize | Inside click=select | "
        "Delete=delete box | Ctrl+D=remove image | Ctrl+Z=undo | Esc=clear selection"
    )

    if not self.image_files:
        self.info_label.setText(f"Open an image folder to begin.\n{shortcuts_text}")
        return

    image_name = self.image_files[self.current_index]
    class_name = CLASS_NAMES[self.current_class_id]

    selected_text = "None"

    if self.selected_box_index is not None and 0 <= self.selected_box_index < len(self.bboxes):
        selected_text = f"{self.selected_box_index}"
    ({CLASS_NAMES[self.bboxes[self.selected_box_index].class_id]})"

    status = (
        f"Image {self.current_index + 1}/{len(self.image_files)}: {image_name} | "
        f"Current class: {self.current_class_id} ({class_name}) | "
        f"Boxes: {len(self.bboxes)} | "
        f"Selected: {selected_text}"
    )

    if extra:
        status += f" | {extra}"

    status += "\n" + shortcuts_text

    self.info_label.setText(status)

def closeEvent(self, event):
    try:
        self.save_labels()
    except Exception:
        pass

    event.accept()

def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())

```

```
if __name__ == "__main__":  
    main()
```

Appendix B: scripts/02_prepare_dataset.py

Purpose: Prepare the raw labeled dataset and generate a ready YOLO dataset.

Main tasks:

- read `raw_dataset/images` and `raw_dataset/labels`
- validate image-label pairs
- remove invalid data
- split data into train/val/test
- generate `data.yaml`

Script:

```
#!/usr/bin/env python3
"""
02_prepare_dataset.py

Prepare a complete YOLO dataset from raw student-labeled folders.

Input:
    extracted_labeled_data/
        student folders...
        images + matching .txt YOLO labels

Output:
    dataset/
        ├── images/
        │   ├── train/
        │   ├── val/
        │   └── test/
        ├── labels/
        │   ├── train/
        │   ├── val/
        │   └── test/
        ├── reports/
        └── data.yaml

Usage:
    python3 02_prepare_dataset.py raw_dataset dataset

Optional:
    python3 02_prepare_dataset.py raw_dataset dataset --train 0.7 --val 0.2 --test 0.1
"""

from __future__ import annotations

import argparse
import random
import shutil
from pathlib import Path
from typing import List, Tuple

import cv2

IMAGE_EXTENSIONS = {
    ".jpg",
    ".jpeg",
    ".png",
    ".bmp",
    ".webp",
}

IGNORE_DIR_NAMES = {
    "__MACOSX",
    ".venv",
    ".venv",
    ".env",
}
```

```

".env",
"Lib",
"Include",
"Scripts",
"site-packages",
"__pycache__",
".git",
".idea",
".vscode",
}

IGNORE_FILE_NAMES = {
    "desktop.ini",
    ".ds_store",
    "label_images.py",
    "README.md",
}

CLASS_NAMES = [
    "meter",
    "window",
    "0",
    "1",
    "2",
    "3",
    "4",
    "5",
    "6",
    "7",
    "8",
    "9",
    "u",
]

def parse_args():
    parser = argparse.ArgumentParser(
        description="Prepare clean YOLO train/val/test dataset from raw labeled data."
    )

    parser.add_argument(
        "source_dir",
        type=Path,
        help="Raw folder containing student-labeled data",
    )

    parser.add_argument(
        "output_dir",
        type=Path,
        help="Output YOLO dataset folder",
    )

    parser.add_argument("--train", type=float, default=0.70)
    parser.add_argument("--val", type=float, default=0.20)
    parser.add_argument("--test", type=float, default=0.10)
    parser.add_argument("--seed", type=int, default=42)

    return parser.parse_args()

def should_ignore_path(path: Path) -> bool:
    parts_lower = {part.lower() for part in path.parts}

    for ignored in IGNORE_DIR_NAMES:
        if ignored.lower() in parts_lower:
            return True

    if path.name.lower() in {name.lower() for name in IGNORE_FILE_NAMES}:
        return True

    return False

def is_image(path: Path) -> bool:
    return path.suffix.lower() in IMAGE_EXTENSIONS

```

```

def collect_files(source_dir: Path):
    images_by_stem = {}
    labels_by_stem = {}

    for path in source_dir.rglob("*"):
        if not path.is_file():
            continue

        if should_ignore_path(path):
            continue

        if is_image(path):
            images_by_stem.setdefault(path.stem, []).append(path)

        elif path.suffix.lower() == ".txt":
            labels_by_stem.setdefault(path.stem, []).append(path)

    return images_by_stem, labels_by_stem

def choose_file(paths):
    return sorted(paths, key=lambda p: str(p).lower())[0]

def validate_label_line(
    line: str,
    label_file: Path,
    line_number: int,
    num_classes: int,
) -> List[str]:
    errors = []
    parts = line.strip().split()

    if len(parts) != 5:
        errors.append(f"{label_file} line {line_number}: expected 5 values, got {len(parts)}")
        return errors

    try:
        class_id = int(parts[0])
        x_center = float(parts[1])
        y_center = float(parts[2])
        width = float(parts[3])
        height = float(parts[4])
    except ValueError:
        errors.append(f"{label_file} line {line_number}: contains non-numeric values")
        return errors

    if not (0 <= class_id < num_classes):
        errors.append(f"{label_file} line {line_number}: invalid class_id {class_id}")

    for name, value in [
        ("x_center", x_center),
        ("y_center", y_center),
        ("width", width),
        ("height", height),
    ]:
        if not (0.0 <= value <= 1.0):
            errors.append(f"{label_file} line {line_number}: {name}={value} outside [0,1]")

    if width <= 0:
        errors.append(f"{label_file} line {line_number}: width must be > 0")

    if height <= 0:
        errors.append(f"{label_file} line {line_number}: height must be > 0")

    x1 = x_center - width / 2
    y1 = y_center - height / 2
    x2 = x_center + width / 2
    y2 = y_center + height / 2

    if x1 < 0 or y1 < 0 or x2 > 1 or y2 > 1:
        errors.append(
            f"{label_file} line {line_number}: box extends outside image "

```

```

        f"{x1={x1:.4f}, y1={y1:.4f}, x2={x2:.4f}, y2={y2:.4f}} "
    )

    return errors

def validate_pair(image_path: Path, label_path: Path) -> List[str]:
    errors = []

    try:
        img = cv2.imread(str(image_path))
    except cv2.error as e:
        errors.append(f"OpenCV failed to read image, possibly too large: {image_path}")
        errors.append(f"OpenCV error: {e}")
        return errors

    if img is None:
        errors.append(f"Unreadable image: {image_path}")
        return errors

    h, w = img.shape[:2]

    if h <= 0 or w <= 0:
        errors.append(f"Invalid image dimensions: {image_path} -> {w}*{h}")

    if label_path.stat().st_size == 0:
        errors.append(f"Empty label file: {label_path}")
        return errors

    with label_path.open("r", encoding="utf-8") as f:
        lines = [line.rstrip("\n") for line in f if line.strip()]

    for line_number, line in enumerate(lines, start=1):
        errors.extend(
            validate_label_line(
                line=line,
                label_file=label_path,
                line_number=line_number,
                num_classes=len(CLASS_NAMES),
            )
        )

    return errors

def prepare_pairs(source_dir: Path, reports_dir: Path):
    images_by_stem, labels_by_stem = collect_files(source_dir)

    image_stems = set(images_by_stem.keys())
    label_stems = set(labels_by_stem.keys())

    matched_stems = sorted(image_stems & label_stems)
    missing_label_stems = sorted(image_stems - label_stems)
    orphan_label_stems = sorted(label_stems - image_stems)

    valid_pairs: List[Tuple[Path, Path, str]] = []

    mapping_lines = []
    missing_label_lines = []
    orphan_label_lines = []
    duplicate_lines = []
    validation_error_lines = []

    for stem in missing_label_stems:
        missing_label_lines.append(
            f"MISSING_LABEL: {stem}\n"
            + "".join(f" image: {p}\n" for p in images_by_stem[stem])
        )

    for stem in orphan_label_stems:
        orphan_label_lines.append(
            f"ORPHAN_LABEL: {stem}\n"
            + "".join(f" label: {p}\n" for p in labels_by_stem[stem])
        )

```

```

clean_index = 1

for stem in matched_stems:
    image_path = choose_file(images_by_stem[stem])
    label_path = choose_file(labels_by_stem[stem])

    if len(images_by_stem[stem]) > 1 or len(labels_by_stem[stem]) > 1:
        duplicate_lines.append(
            f"DUPLICATE STEM: {stem}\n"
            f"  images:\n"
            + "".join(f"    {p}\n" for p in images_by_stem[stem])
            + "  labels:\n"
            + "".join(f"    {p}\n" for p in labels_by_stem[stem])
        )

    errors = validate_pair(image_path, label_path)

    if errors:
        validation_error_lines.append(
            f"INVALID PAIR SKIPPED: {stem}\n"
            f"  image: {image_path}\n"
            f"  label: {label_path}\n"
            + "".join(f"    error: {err}\n" for err in errors)
        )
        continue

    new_stem = f"wm_clean_{clean_index:06d}"
    valid_pairs.append((image_path, label_path, new_stem))

    mapping_lines.append(
        f"{new_stem}\n"
        f"  image_from: {image_path}\n"
        f"  label_from: {label_path}\n"
    )

    clean_index += 1

(reports_dir / "mapping.txt").write_text("\n".join(mapping_lines), encoding="utf-8")
(reports_dir / "missing_labels.txt").write_text("\n".join(missing_label_lines), encoding="utf-8")
(reports_dir / "orphan_labels.txt").write_text("\n".join(orphan_label_lines), encoding="utf-8")
(reports_dir / "duplicates.txt").write_text("\n".join(duplicate_lines), encoding="utf-8")
(reports_dir / "validation_errors.txt").write_text("\n".join(validation_error_lines),
encoding="utf-8")

return valid_pairs, images_by_stem, labels_by_stem, validation_error_lines

def copy_split_pairs(
    pairs: List[Tuple[Path, Path, str]],
    output_dir: Path,
    train_ratio: float,
    val_ratio: float,
    test_ratio: float,
    seed: int,
):
    images_dir = output_dir / "images"
    labels_dir = output_dir / "labels"

    for split in ["train", "val", "test"]:
        (images_dir / split).mkdir(parents=True, exist_ok=True)
        (labels_dir / split).mkdir(parents=True, exist_ok=True)

    random.seed(seed)
    random.shuffle(pairs)

    total = len(pairs)
    train_count = int(total * train_ratio)
    val_count = int(total * val_ratio)

    train_pairs = pairs[:train_count]
    val_pairs = pairs[train_count:train_count + val_count]
    test_pairs = pairs[train_count + val_count:]

    splits = {
        "train": train_pairs,

```



```

        "val": val_pairs,
        "test": test_pairs,
    }

    for split_name, split_pairs in splits.items():
        for image_path, label_path, new_stem in split_pairs:
            new_image_name = new_stem + image_path.suffix.lower()
            new_label_name = new_stem + ".txt"

            shutil.copy2(image_path, images_dir / split_name / new_image_name)
            shutil.copy2(label_path, labels_dir / split_name / new_label_name)

    return {
        "total": total,
        "train": len(train_pairs),
        "val": len(val_pairs),
        "test": len(test_pairs),
    }

def create_data_yaml(output_dir: Path):
    data_yaml = output_dir / "data.yaml"

    dataset_path = output_dir.resolve()

    lines = [
        f"path: {dataset_path}",
        "train: images/train",
        "val: images/val",
        "test: images/test",
        "",
        "names:",
    ]

    for i, name in enumerate(CLASS_NAMES):
        lines.append(f" {i}: {name}")

    data_yaml.write_text("\n".join(lines) + "\n", encoding="utf-8")

def clean_output_dir(output_dir: Path):
    if output_dir.exists():
        shutil.rmtree(output_dir)

    output_dir.mkdir(parents=True, exist_ok=True)

def main():
    args = parse_args()

    total_ratio = args.train + args.val + args.test
    if abs(total_ratio - 1.0) > 1e-9:
        raise ValueError("Train, val, and test ratios must add up to 1.0")

    source_dir = args.source_dir
    output_dir = args.output_dir

    if not source_dir.exists():
        raise FileNotFoundError(f"Source folder not found: {source_dir}")

    clean_output_dir(output_dir)

    reports_dir = output_dir / "reports"
    reports_dir.mkdir(parents=True, exist_ok=True)

    valid_pairs, images_by_stem, labels_by_stem, validation_error_lines = prepare_pairs(
        source_dir=source_dir,
        reports_dir=reports_dir,
    )

    if not valid_pairs:
        print("No valid image-label pairs found. Dataset was not created.")
        return

    split_counts = copy_split_pairs(

```

```

        pairs=valid_pairs,
        output_dir=output_dir,
        train_ratio=args.train,
        val_ratio=args.val,
        test_ratio=args.test,
        seed=args.seed,
    )

    create_data_yaml(output_dir)

    print("\n===== YOLO DATASET PREPARATION COMPLETE =====")
    print(f"Source folder: {source_dir}")
    print(f"Output folder: {output_dir}")
    print()
    print(f"Images found: {sum(len(v) for v in images_by_stem.values())}")
    print(f"Labels found: {sum(len(v) for v in labels_by_stem.values())}")
    print(f"Valid image-label pairs used: {split_counts['total']}")
    print(f"Invalid pairs skipped: {len(validation_error_lines)}")
    print()
    print("Split:")
    print(f"Train: {split_counts['train']}")
    print(f"Val: {split_counts['val']}")
    print(f"Test: {split_counts['test']}")
    print()
    print("Created:")
    print(f" {output_dir} / 'images' / 'train'")
    print(f" {output_dir} / 'images' / 'val'")
    print(f" {output_dir} / 'images' / 'test'")
    print(f" {output_dir} / 'labels' / 'train'")
    print(f" {output_dir} / 'labels' / 'val'")
    print(f" {output_dir} / 'labels' / 'test'")
    print(f" {output_dir} / 'data.yaml'")
    print(f" {reports_dir}")
    print()
    print("\n✓ READY FOR TRAINING")

if __name__ == "__main__":
    main()

```

Appendix C: scripts/03_train_yolo.py

Purpose: Train the first YOLO model using dataset/data.yaml.

Main tasks:

- load YOLO weights
- train using prepared dataset
- save results in `training_runs/`

Script:

```
#!/usr/bin/env python3
"""
03_train.py

Train a YOLO model using Ultralytics.

Install first: pip install ultralytics

Usage:
    python3 scripts/03_train.py

Or:
    python3 scripts/03_train.py --data dataset/data.yaml --model yolov8n.pt --epochs 100 --imgsz 640
    --batch 8 --name water_meter_corrected_names
"""

from __future__ import annotations

import argparse
from pathlib import Path

from ultralytics import YOLO

def parse_args():
    parser = argparse.ArgumentParser(
        description="Train YOLO model for water meter detection and reading."
    )

    parser.add_argument("--model", default="yolov8n.pt", help="YOLO model weights")
    parser.add_argument("--data", default="dataset/data.yaml", help="Path to data.yaml")
    parser.add_argument("--epochs", type=int, default=100, help="Number of training epochs")
    parser.add_argument("--imgsz", type=int, default=640, help="Image size")
    parser.add_argument("--batch", type=int, default=8, help="Batch size")
    parser.add_argument("--project", default="training_runs", help="Output project folder")
    parser.add_argument("--name", default="water_meter_yolov8n", help="Experiment name")

    return parser.parse_args()

def main() -> None:
    args = parse_args()

    data_yaml = Path(args.data)

    if not data_yaml.exists():
        raise FileNotFoundError(f"data.yaml not found: {data_yaml}")

    model = YOLO(args.model)

    model.train(
        data=str(data_yaml),
        epochs=args.epochs,
        imgsz=args.imgsz,
        batch=args.batch,
        project=args.project,
        name=args.name,
    )

if __name__ == "__main__":
    main()
```

Appendix D: scripts/04_predict.py

Purpose: Run prediction on test images using the latest trained best.pt model.

Main tasks:

- find latest `best.pt`
- predict on `dataset/images/test`
- save annotated images
- save prediction text files

Script:

```
#!/usr/bin/env python3
"""
04_predict.py

Run YOLO inference on test images and save visualized predictions.

Fully automatic:
- Finds latest best.pt from training_runs/
- Uses dataset/images/test
- Saves annotated images
- Saves prediction .txt files
- Saves confidence scores

Usage:
    python3 scripts/04_predict.py
"""

from __future__ import annotations

from pathlib import Path
from typing import Iterable

from ultralytics import YOLO

TRAINING_ROOT = Path("training_runs")
TEST_IMAGES_DIR = Path("dataset/images/test")
OUTPUT_PROJECT = Path("prediction_outputs")
OUTPUT_NAME = "test_predictions"

IMAGE_SIZE = 640
CONFIDENCE = 0.25

SAVE_TXT = True
SAVE_CONF = True

VALID_IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

def find_latest_best_model(training_root: Path) -> Path:
    best_models = list(training_root.glob("*/weights/best.pt"))

    if not best_models:
        raise FileNotFoundError(
            f"No best.pt model found inside: {training_root}"
        )

    return max(best_models, key=lambda p: p.stat().st_mtime)

def iter_test_images(folder: Path) -> Iterable[Path]:
    for path in sorted(folder.iterdir()):
        if path.is_file() and path.suffix.lower() in VALID_IMAGE_EXTS:
            yield path

def main() -> None:
```

```

model_path = find_latest_best_model(TRAINING_ROOT)

if not TEST_IMAGES_DIR.exists():
    raise FileNotFoundError(
        f"Test images directory not found: {TEST_IMAGES_DIR}"
    )

test_images = list(iter_test_images(TEST_IMAGES_DIR))

if not test_images:
    raise RuntimeError(f"No supported test images found in: {TEST_IMAGES_DIR}")

OUTPUT_PROJECT.mkdir(parents=True, exist_ok=True)

print("===== YOLO TEST PREDICTION =====")
print(f"Model: {model_path}")
print(f"Test folder: {TEST_IMAGES_DIR}")
print(f"Images: {len(test_images)}")
print(f"Output root: {OUTPUT_PROJECT}")
print(f"Run name: {OUTPUT_NAME}")
print(f"Image size: {IMAGE_SIZE}")
print(f"Confidence: {CONFIDENCE}")
print()

model = YOLO(str(model_path))

results = model.predict(
    source=str(TEST_IMAGES_DIR),
    imgsz=IMAGE_SIZE,
    conf=CONFIDENCE,
    save=True,
    save_txt=SAVE_TXT,
    save_conf=SAVE_CONF,
    show_labels=True,
    show_conf=False,
    project=str(OUTPUT_PROJECT),
    name=OUTPUT_NAME,
    exist_ok=False,
)

output_dir = OUTPUT_PROJECT / OUTPUT_NAME
label_dir = output_dir / "labels"

print("\nPrediction completed. [✓]")
print(f"Annotated images saved in: {output_dir}")
print(f"Prediction txt files saved in: {label_dir}")

print("\nPer-image summary:")
for image_path, result in zip(test_images, results):
    num_boxes = 0 if result.bboxes is None else len(result.bboxes)
    print(f" - {image_path.name}: {num_boxes} detection(s)")

if __name__ == "__main__":
    main()

```

Appendix E: scripts/05_retrain.py

Purpose: Retrain the model starting from the latest best.pt model.

Main tasks:

- find latest **best.pt**
- continue training
- save a new training run

Script:

```
#!/usr/bin/env python3
"""
05_retrain.py

Retrain / continue training YOLO using the latest best.pt model found in training_runs/.

Fully automatic:
- Finds latest training_runs/*/weights/best.pt
- Uses dataset/data.yaml
- Creates a new training run
- Can be run again and again after many training runs

Usage:
    python3 scripts/05_retrain.py
"""

from __future__ import annotations

from datetime import datetime
from pathlib import Path

from ultralytics import YOLO

TRAINING_ROOT = Path("training_runs")
DATA_YAML = Path("dataset/data.yaml")

EPOCHS = 10
IMAGE_SIZE = 640
BATCH_SIZE = 8

PROJECT_NAME = "training_runs"
RUN_PREFIX = "water_meter_retrain"

def find_latest_best_model(training_root: Path) -> Path:
    best_models = list(training_root.glob("*/weights/best.pt"))

    if not best_models:
        raise FileNotFoundError(
            f"No best.pt model found inside: {training_root}"
        )

    return max(best_models, key=lambda p: p.stat().st_mtime)

def make_run_name(prefix: str) -> str:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    return f"{prefix}_{timestamp}"

def main() -> None:
    if not DATA_YAML.exists():
        raise FileNotFoundError(f"data.yaml not found: {DATA_YAML}")

    latest_best = find_latest_best_model(TRAINING_ROOT)
    run_name = make_run_name(RUN_PREFIX)
```

```
print("=====YOLO RETRAINING =====")
print(f"Starting from: {latest_best}")
print(f>Data YAML:      {DATA_YAML}")
print(f"New run name:    {run_name}")
print(f"Epochs:          {EPOCHS}")
print(f"Image size:        {IMAGE_SIZE}")
print(f"Batch size:        {BATCH_SIZE}")
print()

model = YOLO(str(latest_best))

model.train(
    data=str(DATA_YAML),
    epochs=EPOCHS,
    imgsz=IMAGE_SIZE,
    batch=BATCH_SIZE,
    project=PROJECT_NAME,
    name=run_name,
)

print("\nRetraining completed. [✓]")
print(f"New run saved under: {PROJECT_NAME}/{run_name}")

if __name__ == "__main__":
    main()
```